Database systems 1

dr hab. inż. Przemysław Korytkowski, prof. ZUT

Lecture 11 – Data warehouses and business intelligence

1

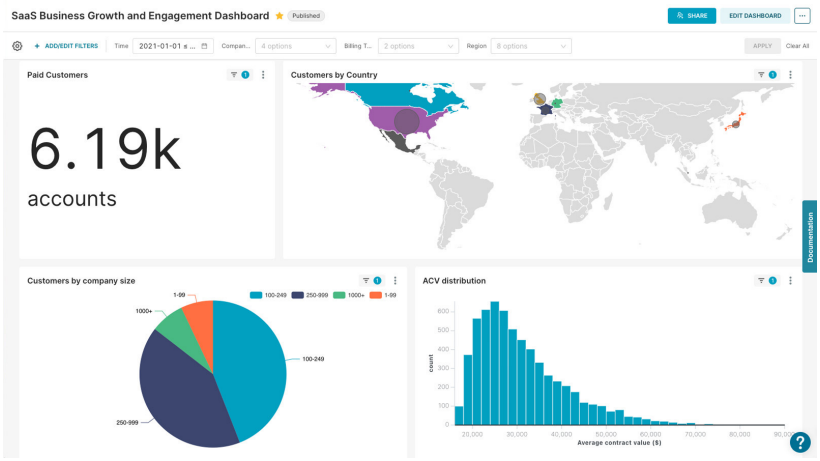
1

Grafana – monit-grafana-open.cern.ch

2

2

Apache Superset

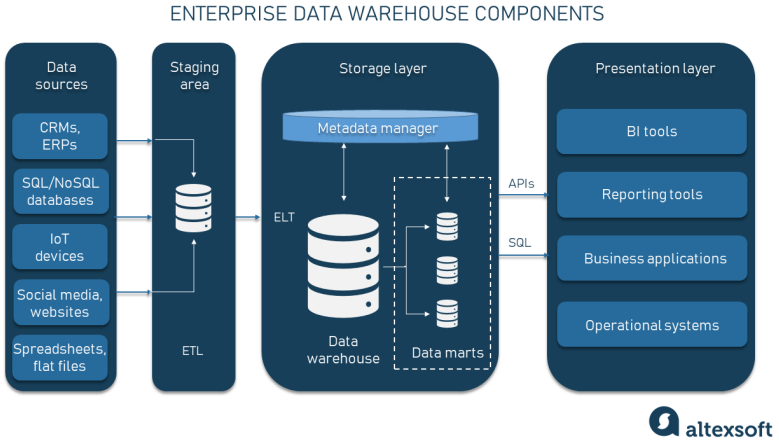


source: https://superset.apache.org

Transactions versus analyses

- System administrators are reluctant to grant business users (analysts) access to database systems, fearing that complex ad hoc queries will overload the servers.
- In addition, business users often want to see historical trends and combine data from multiple databases.
- For these reasons, almost every company has created a large data warehouse and periodically ‘scraped’ data from operational systems into it.
- Business users could then perform complex ad hoc queries on the data in the warehouse without affecting other users.

Architecture of a data warehouse



5

5

OLTP vs. DW vs. OLAP

- OLTP (**Online Transaction Processing**)
  - OLTP systems are operational databases designed for speed and integrity. They handle a large volume of short, real-time transactions.
  - Focus: Immediate consistency (ACID compliance) and low-latency writes.
  - Examples: PostgreSQL, SQL Server, and MySQL.
  - Use Case: ATM withdrawals, e-commerce checkouts, and hotel bookings.
- DW (**Data Warehouse**)
  - A DW is a central repository that integrates data from multiple OLTP sources for long-term storage and reporting.
  - Focus: Subject-oriented, historical context, and non-volatile data storage.
  - Role: Acts as the foundation for OLAP tools.
  - Examples: Amazon Redshift, Google BigQuery, and Teradata.
- OLAP (**Online Analytical Processing**)
  - OLAP refers to the analytical techniques used to extract insights from data warehouses or specialized databases.
  - Focus: Complex aggregations (e.g., "Total sales by region and quarter").
  - Mechanism: Uses multidimensional analysis (slicing and dicing).
  - Trends in 2025: Modern engines like ClickHouse blur the lines by supporting real-time OLAP without traditional "cubes".

6

6

Contrasting OLTP and OLAP

Workload

- Data warehouses are designed to accommodate ad hoc queries and data analysis
- Data warehouse should be optimized to perform well for a wide variety of possible query and analytical operations.
- OLTP systems support only predefined operations. Applications might be specifically tuned or designed to support only these operations.

Data modifications

- A data warehouse is updated on a regular basis by the ETL process (run nightly or weekly) using bulk data modification techniques. The end users of a data warehouse do not directly update the data warehouse.
- In OLTP systems, end users routinely issue individual data modification statements to the database. The OLTP database is always up to date, and reflects the current state of each business transaction.

Source: Oracle Database Data Warehousing Guide, 21c 7

7

Contrasting OLTP and OLAP

Schema design

- OLAP often use partially denormalized schemas to optimize query and analytical performance.
- OLTP systems often use fully normalized schemas to optimize update/insert/delete performance, and to guarantee data consistency.

Typical operations

- A typical data warehouse query scans thousands or millions of rows. For example, "Find the total sales for all customers last month."
- A typical OLTP operation accesses only a handful of records. For example, "Retrieve the current order for this customer."

Historical data

- Data warehouses usually store many months or years of data. This is to support historical analysis and reporting.
- OLTP systems usually store data from only a few weeks or months. The OLTP system stores only historical data as needed to successfully meet the requirements of the current transaction.

Source: Oracle Database Data Warehousing Guide, 21c 8

8

Data warehouse – characteristics (by Inmon)

- **Subject-Oriented**
  - a DW is organized around major subjects of the business. Examples: Sales, Products, Customers, or Claims.
  - This allows users to look at a high-level subject across the entire organization rather than looking at how a specific application handles that data.
- **Integrated**
  - Data from disparate sources must be standardized before it is loaded.
  - Consistency: Converting different date formats (DD/MM/YYYY vs. MM/DD/YYYY), standardizing currency (converting everything to USD), and resolving naming conflicts (e.g., one system calls it "Client_ID" and another "Cust_No").
  - Physical Integration: Data is physically moved into a central store (like Snowflake or Amazon Redshift) to allow for high-performance joins that are impossible across isolated silos.

9

9

Data warehouse – characteristics (by Inmon)

- **Time-Variant**
  - A DW stores the history of that data. Every record in a DW is associated with a specific point in time.
  - Trend Analysis: This allows for "Slicing by Time," enabling analysts to compare this quarter's performance to the same quarter five years ago.
  - Techniques: Specialized methods like Slowly Changing Dimensions (SCD) are used to track how data values change over months or years.
- **Non-Volatile**
  - In a DW, data is permanent once it is entered.
  - Read-Only Focus: While data is periodically added (appended), it is almost never changed or deleted by the end-user.
  - Data Integrity: This ensures that if you run a report today for "Sales in 2023," you will get the exact same result if you run it again in 2026. This stability is essential for auditing and long-term strategic planning.

10

10

The "Single Source of Truth" (SSOT)

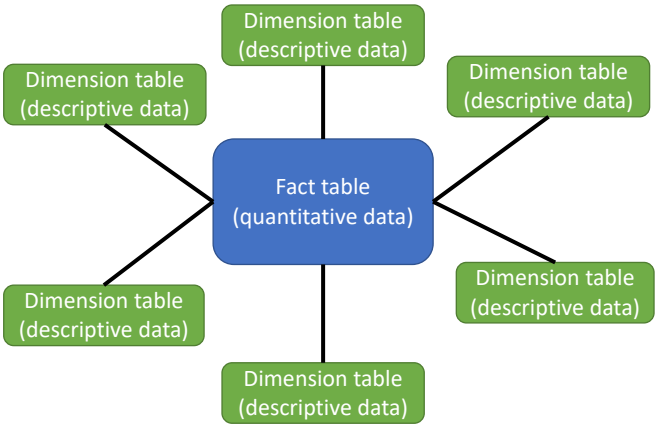
- In a modern enterprise, data is fragmented across hundreds of SaaS applications (Salesforce, Zendesk), internal databases (ERP, HR), and streaming logs.
- The Problem: Without a DW, different departments report different numbers for the same metric (e.g., Marketing counts "leads" differently than Sales).
- The DW Solution: The DW acts as the authoritative data source. By centralizing data into a single repository with governed definitions, it ensures that every stakeholder—from a data scientist to a CEO—is looking at the same synchronized, validated information.

11

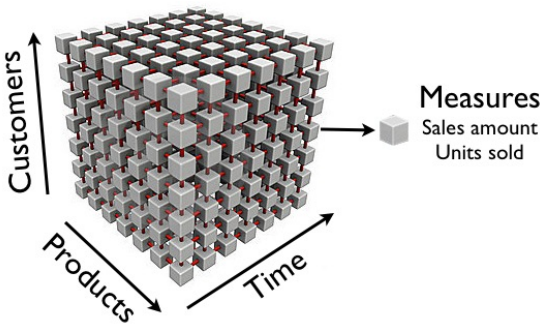
11

Data Model in Data Warehouses

Relational OLAP (ROLAP)
Star Model



Multi dimensional OLAP (MOLAP)
Data cube (hypercube)



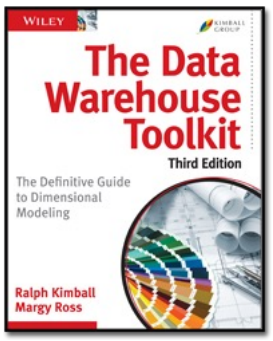
source: eazybi.com

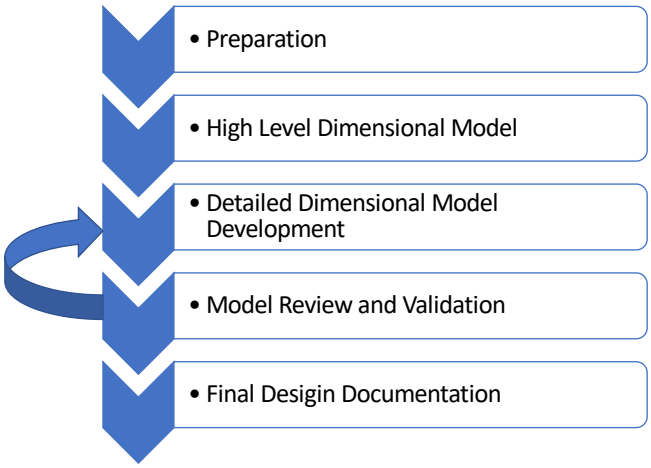
12

12

Dimensional Modeling Techniques

- First edition 1996 r.





13

13

Star Model

CategoryID	Name
Cat_a	Cutlery
Cat_b	Stationery

CusID	ProductID	CategoryID	Quantity
111	abc	Cat_a	15
222	cde	Cat_a	2
333	xyz	Cat_b	3
111	xyz	Cat_b	7
222	abc	Cat_a	5
111	cde	Cat_a	9
333	abc	Cat_a	12

CusID	FirstName	LastName
111	Anna	Nowak
222	Jan	Kowalski
333	Ewa	Wiśniewska

ProductID	Name	Producer
abc	fork	Land-Pol
cde	knife	PPPN
xyz	pencil	ZPPO

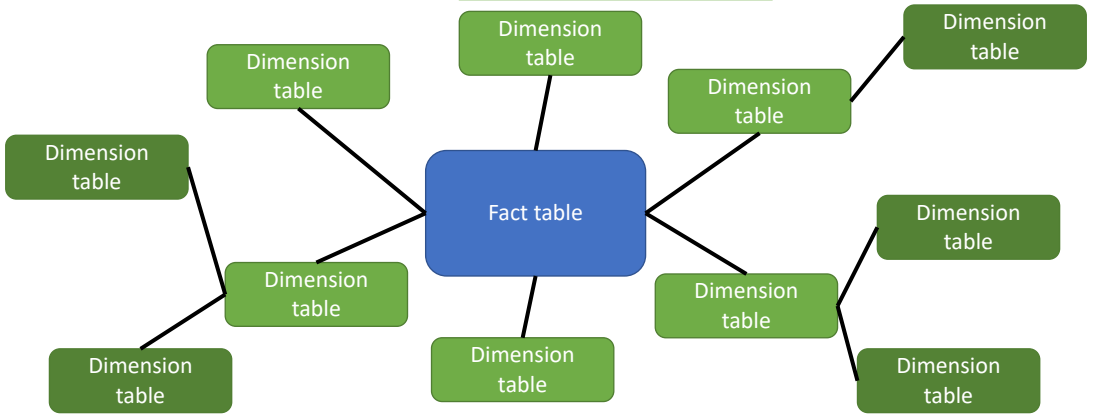
14

14

Snowflake model

Pros:
- Reduced redundancy
- Better data consistency
- Easier governance
- Better support for slowly changing dimensions

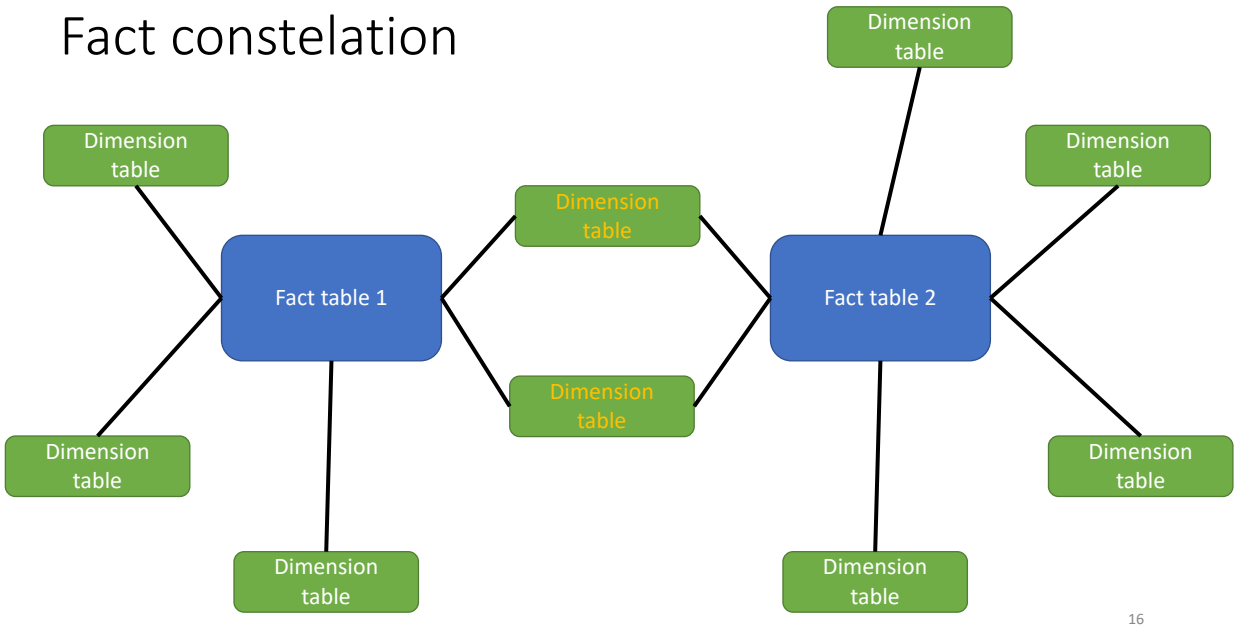
Cons:
- More complex queries
- More joins → slower queries
- Harder for business users
- Less intuitive BI modeling



15

15

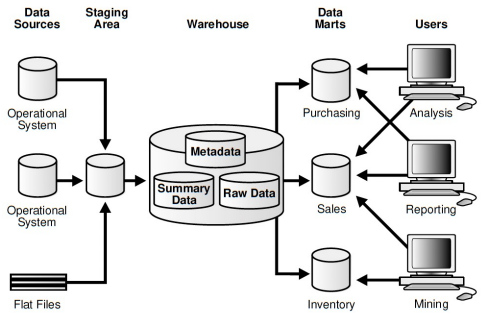
Fact constelation



16

16

Data mart



Data warehouse:

- Covers many subject areas
- Stores highly detailed information
- Works to integrate all data sources
- Does not necessarily use a dimensional model, but feeds dimensional models.

Data mart:

- Often contains only one subject area – for example, finance or sales
- May contain more summarised data (although it may contain full details)
- Focuses on integrating information from a specific subject area or set of source systems
- Is built on a dimensional model using a star schema.

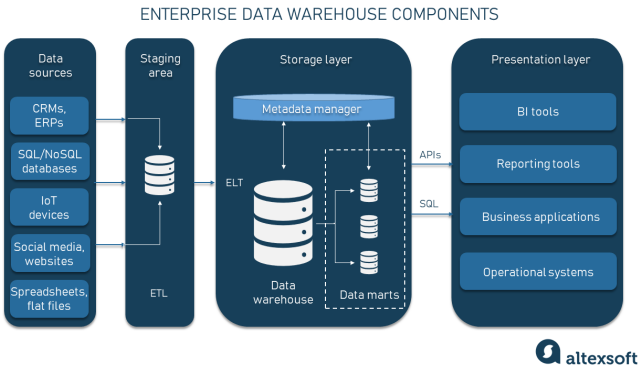
17

17

ETL - Extract, Transform, Load

Repeatable, programmed data flow.

- **Extract** – obtaining data from source operating or archival systems, which are the primary source of data for data warehouses
- **Transform** – transforming data – which may include cleaning, filtering, validating, enriching and applying business rules
- **Load** – loading data into a data warehouse or other database or application containing data.



18

18

Extract

- Extraction is the process of reading data from source systems and moving it to a staging (raw layer) without changing its business meaning.
- Extraction should be minimally invasive to source systems.
- Goals of the Extract Step:
  - Obtain a complete and correct copy of source data
  - Minimize impact on operational systems
  - Ensure repeatability and recoverability
  - Preserve lineage and auditability
  - Enable incremental loading

19

19

Extract – types of source systems

- Operational Databases (OLTP): PostgreSQL, Oracle, SQL Server, MySQL
  - Highly normalized
  - High transaction rates
- Files: CSV, JSON, XML, Excel, Logs
- APIs / SaaS
  - CRM (Salesforce)
  - Web analytics
  - Payment systems
- Streaming Sources
  - Kafka
  - Event logs
  - IoT devices

20

20

Extraction strategies

A) Full Extraction

- Entire source table extracted every time
- Pros
  - Simple
  - No dependency on source metadata
- Cons
  - Slow
  - High load on source
  - Not scalable
- When to use
  - Small tables
  - Reference data
  - Initial loads

B) Incremental Extraction

- Only new or changed data is extracted.
- Timestamp-based Extraction
- Surrogate Key / ID-based
- Change Data Capture (CDC)
  - Capture changes from database logs

C) Event-based Extraction

- Each business event is emitted
- Data warehouse consumes events
- Used in modern streaming architectures

21

21

Extraction Architecture

Pull-based Extraction

- Periodic polling
- Common in batch ETL

Push-based Extraction

- Event-driven
- Near real-time

Staging (Raw Layer)

- Rationale:
  - Isolation from transformations
  - Restartability
  - Auditing
- Staging table principles
  - Same structure as source
  - No business logic
  - Include technical metadata

22

22

Extraction Architecture

Data Quality at Extract Stage

- What NOT to do
  - Business transformations
  - Aggregations
- What TO do
  - Schema validation
  - Type checks
  - Row count checks
  - Source–target reconciliation

Error Handling & Recovery

- Common failures
  - Network errors
  - Partial loads
  - Schema changes
- Best practices
  - Idempotent extracts
  - Checkpointing
  - Retry logic
  - Dead-letter queues (for streaming)

23

23

Transform

Transformation is the process of converting raw, extracted data into a consistent, trustworthy, analytics-ready form aligned with:

- Business definitions
- Data models (facts & dimensions)
- Reporting requirements
- Transformation encodes business logic.

24

24

Transform

Structural Transformations (Low-level)

- Data Type Conversion
  - CAST(order_date AS DATE)
- Column Renaming
  - order_total → sales_amount
- Schema Alignment
  - Adding missing attributes
  - Dropping unused attributes

Enables integration across sources

Data Cleansing Transformations

- Handling Missing Values
  - Default values
  - NULL propagation
  - Domain-specific rules
- Standardization
  - "PL", "Pol" → "Poland"
- Deduplication
  - Same entity appears multiple times
  - Methods
    - Natural key matching
    - Fuzzy matching (advanced)

25

25

Transform

Business Rule Transformations

- Example: Revenue Calculation
  - revenue = price x quantity - discount
- Business rules must be:
  - Explicit
  - Documented
  - Consistent

Conforming Dimensions (Integration Logic)

- Same concept, different sources
  - 'F', 'Female', 'Ms.' → 'Female'

Slowly Changing Dimensions (SCD)

- SCD Type 1 – Overwrite
  - Old value disappears
- SCD Type 2 – History Tracking
  - ID, value, valid_from, valid_to
- SCD Type 3 – Limited History
  - ID, value, old_value

26

26

Transform

Data Quality Checks

- Validation rules
  - NOT NULL constraints
  - Range checks
  - Referential integrity
- Reject vs quarantine
  - Bad rows sent to error tables
  - Pipeline continues

Common Anti-Patterns

- Business logic hidden in BI tools
- Hard-coded values
- Mixing extraction with transformation
- No documentation

27

27

Load

- Loading is the process of writing transformed data into the target analytical structures, typically:
- Data warehouse tables (facts & dimensions)
- Data marts
- Curated layers (Gold)
- Load must be correct, consistent, and recoverable.

- Load is where data becomes official
- Correct sequencing matters
- Dimensions are harder than facts
- Idempotency is essential
- Good loading enables reliable BI

28

28

Load

Load Targets

- Dimension Tables
  - Customers
  - Products
  - Time
  - Geography
- Fact Tables
  - Sales
  - Orders
  - Transactions
  - Events
- Aggregate Tables / Data Marts
  - Daily sales
  - Monthly KPIs

Load Types

- Initial (Full) Load
  - First population of a table
  - Large volume
  - Often executed once
- Incremental Load
  - Insert-only Loads
    - Append new records
    - Common for fact tables
  - Insert + Update Loads
    - Typical for dimensions

```
MERGE INTO dim_customer d
USING staging_customer s
ON d.customer_id = s.customer_id
WHEN MATCHED THEN UPDATE
WHEN NOT MATCHED THEN INSERT;
```

29

29

Load

SCD Handling During Load

- SCD Type 1 (Overwrite)
  - UPDATE existing row
- SCD Type 2 (History)
  - Close old record
  - Insert new record
- Load logic
  - Find changed rows
  - Set valid_to on old row
  - Insert new row with new surrogate key

Close current record

```
UPDATE dim_customer d
SET valid_to = CURRENT_DATE - 1, current = FALSE
FROM stg_customer s
WHERE d.customer_id = s.customer_id
  AND d.current = TRUE
  AND d.city <> s.city;
```

Insert new record

```
INSERT INTO dim_customer
(customer_id, city, valid_from, valid_to, current)
SELECT customer_id, city, CURRENT_DATE, '9999-12-31', TRUE
FROM stg_customer s
WHERE EXISTS (
  SELECT 1
  FROM dim_customer d
  WHERE d.customer_id = s.customer_id
    AND d.current = FALSE );
```

30

30

Load

Load Performance Techniques

- Bulk loading
  - COPY commands
  - Batch inserts
- Partitioning
  - By date
  - By business key
- Parallelism
  - Load multiple partitions simultaneously

Common Anti-Patterns

- Loading facts before dimensions
- Overwriting fact history
- No surrogate keys
- No reconciliation checks
- Silent failures

31

31

ETL vs ELT

ELT is used when transformation is better done inside the target analytical system than before loading the data.

This shift is driven by cloud data platforms, cheap storage, and powerful SQL engines.

- Load first, transform later
- Raw data is an asset
- ELT increases flexibility and agility
- ETL is still relevant in regulated environments

Aspect	ETL	ELT
Transform location	Before DW	Inside DW
Language	Tool-specific	SQL
Compute	ETL server	DW engine
Storage cost	Higher	Lower
Flexibility	Low	High
Reprocessing	Difficult	Easy
Scalability	Limited	High

32

32

ETL tools

- Traditional ETL Tools (Batch-oriented, Pre-load Transform)
  - IBM InfoSphere DataStage
  - SAP BusinessObjects Data Integrator / SAP Data Services
  - Microsoft SSIS (SQL Server Integration Services)
- Cloud-Native Data Integration Platforms
  - AWS Glue
  - Azure Data Factory
  - Databricks
- Lightweight / Scriptable Frameworks
  - Apache Airflow (Orchestration + workflow (ETL/ELT), DAG-based workflow patterns, scheduling)
  - dbt (Data Build Tool) - Transformation framework

33

33

Data warehouses evolution

The Era of On-Premise Silos (1980s–2000s)

- Early warehouses were built on architecture, where a single physical node managed both storage and compute.
- Physical Constraints: Scaling required significant capital expenditure to purchase and install new hardware.
- Operational Silos: Data was often locked in isolated departmental systems, making cross-functional analysis difficult and leading to "data silos".
- Maintenance Burden: IT teams were responsible for manual software upgrades, security patches, and hardware troubleshooting.

34

34

Data warehouses evolution

Transition to Cloud Data Warehouses (2010s–Present)

- The rise of Big Data and IoT created a demand for instant scalability that physical infrastructure could not meet. Modern Cloud Data Warehouses (CDW) introduced fundamental architectural changes.
- Separation of Compute and Storage: Platforms like Snowflake pioneered this, allowing users to scale processing power and storage capacity independently and instantly.
- Consumption-Based Pricing: The shift moved costs from CapEx to OpEx (pay-as-you-go), where companies only pay for the exact resources they consume.
- Serverless Operations: Modern solutions abstract infrastructure management entirely, automatically optimizing query execution without manual intervention.

35

35

SQL for analytics – ROLLUP and CUBE

```
MySQL
SELECT <attributes>
FROM   <one or more relations>
WHERE <conditions>
GROUP BY <attributes> WITH ROLLUP;
```

```
SQL Server
SELECT <attributes>
FROM   <one or more relations>
WHERE <conditions>
GROUP BY [CUBE, ROLLUP] <attributes>;
```

```
SQL Server
SELECT <attributes>
FROM   <one or more relations>
WHERE <conditions>
GROUP BY GROUPING SETS (
    (attr1, attr2),
    (attr1),
    ()
);
```

36

36

SQL for analytics

```
SELECT CustID, ProdID, CatID, Quantity
FROM   sales
GROUP BY CatID, ProdID, CustID WITH ROLLUP;
```

```
SELECT CustID, ProdID,
       CatID, Quantity
FROM   sales;
```

CusID	ProductID	CategoryID	Quantity
111	abc	Cat_a	15
222	cde	Cat_a	2
333	xyz	Cat_b	3
111	xyz	Cat_b	7
222	abc	Cat_a	5
111	cde	Cat_a	9
333	abc	Cat_a	12

CustID	ProductID	CategoryID	Quantity
111	abc	Cat_a	15
222	cde	Cat_a	2
333	xyz	Cat_b	3
111	xyz	Cat_b	7
222	abc	Cat_a	5
111	cde	Cat_a	9
333	abc	Cat_a	12
NULL	abc	Cat_a	32
NULL	NULL	Cat_b	10
…	…	…	
NULL	NULL	NULL	53

37

37

SQL for analytics – window functions

```
SELECT CustID, ProdID, CatID, Quantity,
       SUM(Quantity) OVER()  AS Total,
       SUM(QUANTITY) OVER(PARTITION BY CustID) AS Total_by_Cust
       RANK() OVER(PARTITION BY CustID, ORDER BY CatID) AS rank
FROM   sales;
```

CusID	ProductID	CategoryID	Quantity	Total	Total_by_Cus	rank
111	abc	Cat_a	15	53	31	1
222	cde	Cat_a	2	53	7	1
333	xyz	Cat_b	3	53	15	2
111	xyz	Cat_b	7	53	31	3
222	abc	Cat_a	5	53	7	1
111	cde	Cat_a	9	53	31	2
333	abc	Cat_a	12	53	15	1

38

38

SQL for analytics – window functions

```
SELECT CustID, ProdID, CatID, Quantity,
       SUM(Quantity) OVER()  AS Total,
       SUM(QUANTITY) OVER(PARTITION BY CusID) AS Total_by_Cus
FROM   sales;
```

```
SELECT CustID, ProdID, CatID, Quantity,
       SUM(Quantity) OVER()  AS Total,
       SUM(QUANTITY) OVER w AS Total_by_Cus
FROM   sales
WINDOW w AS (PARTITION BY CusID);
```

39

39

SQL for analytics – PIVOT

```
Oracle
SELECT * FROM
(
      SELECT CustID,
      FROM   sales
)
PIVOT
(
      SUM(Quantity)
      FOR CategoryID IN (Cat_a, Cat_b)
)
ORDER BY CustID;
```

CustID	ProductID	CategoryID	Quantity
111	abc	Cat_a	15
222	cde	Cat_a	2
333	xyz	Cat_b	3
111	xyz	Cat_b	7
222	abc	Cat_a	5
111	cde	Cat_a	9
333	abc	Cat_a	12

CustID	Cat_a	Cat_b
111	24	7
222	7	0
333	3	12

40

40

SQL for analytics - Views

```
SQL Server
CREATE MATERIALIZED VIEW viewName AS
      SELECT <attributes>
      FROM <table>
      JOIN <table> ON <conditions>
      …
      GROUP BY <attributes>
      ORDER BY <attributes>;
```

41

41

Business Intelligence (BI)

- **Data Mining & Analysis**: Sifting through vast datasets to identify hidden patterns and trends using machine learning.
- **Reporting & Visualization**: Converting complex data into clear, graphical narratives (dashboards) that anyone can understand.
- **Performance Management**: Monitoring Key Performance Indicators (KPIs) against business goals.
- **Predictive Analytics**: Using historical data to forecast future outcomes and market changes.

42

42

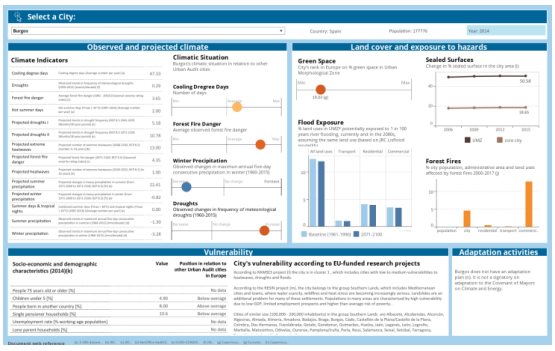
Business Intelligence (BI)

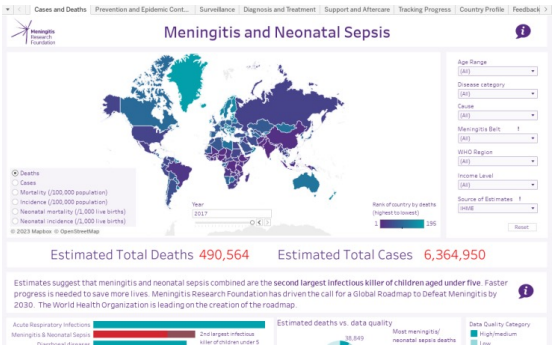
- Conversational Analytics (NLP): Users can ask questions in plain English (e.g., "Why did sales drop in March?") and receive instant visualizations or narrative summaries.
- Automated Insights: Machine learning (ML) identifies hidden patterns, anomalies, and trends automatically, often suggesting "Why" something happened before a user even asks.
- Predictive & Prescriptive Modeling: Tools no longer just show historical data; they forecast future outcomes (predictive) and recommend specific actions to take (prescriptive).
- Drag-and-Drop Interfaces: Users can build complex dashboards without writing code or SQL.
- Data Storytelling: Modern tools automatically generate written narratives alongside charts to explain the "story" behind the numbers, making reports easier for executives to digest.
- Mobile-First BI: Fully functional mobile apps allow teams to access real-time dashboards and receive automated alerts on the go.

43

Business intelligence – tableau

Tableau is a visual analytics platform transforming the way we use data to solve problems—empowering people and organizations to make the most of their data.





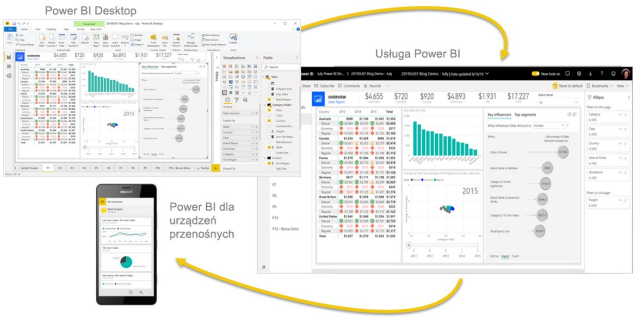
Source: tableau.com

Business intelligence – Power BI

Power BI is a suite of software services, applications, and connectors that work together to transform unrelated data sources into consistent, visually appealing, and interactive analyses.

The data can be an Excel spreadsheet or a collection of hybrid cloud-based and local data warehouses.

Power BI makes it easy to connect to data sources, visualise and discover insights, and share them with everyone or select individuals.



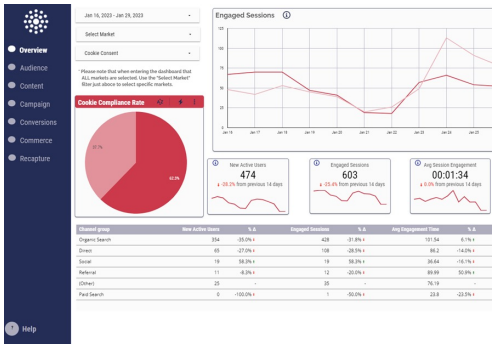
Source: Microsoft

45

45

Business intelligence – Looker

Looker is Google for your business data. More than just a modern business intelligence platform, you can turn to Looker for self-service or governed BI, build your own custom applications with trusted metrics, or even bring Looker modeling to your existing BI environment.

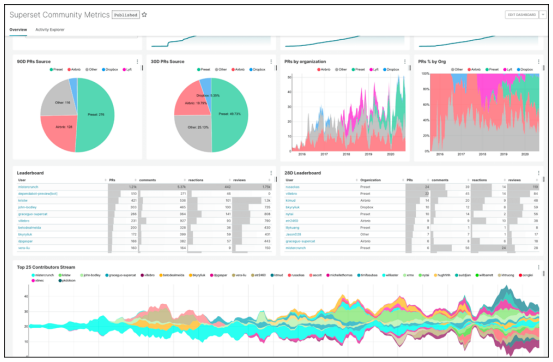


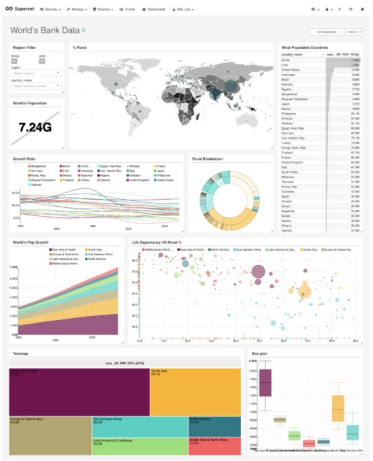
source: Google

46

Apache Superset - superset.apache.org/

Superset is a modern data exploration and data visualization platform. Superset can replace or augment proprietary business intelligence tools for many teams. Superset integrates well with a variety of data sources.





source: Apache Superset

Dziękuję za uwagę